Kotlin Backend Developer.Professional

REC **Проверить, идет ли запись**

Меня хорошо видно && слышно?

Правила вебинара

Активно участвуем

Off-topic обсуждаем в OTUS-Kotlin-2023-12

Задаем вопрос в чат или голосом

Вопросы вижу в чате, могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или задайте вопрос

Тема вебинара

Общие принципы защиты информации

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП	DSL	
Модуль 2: Расширенные возможности Kotlin	Корутины	Sequence, Flow	Multiplatform	Interop	Gradle	
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	DevOps	Архитектура		
Модуль 4: Разработка API	OpenAPI	CollabAPI	Модели и маперы	Testing	Monitoring	
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Репози- торий	Конкурен- ция	Postgres	Cassandra	Gremlin	
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде		
Модуль 9: Проектная работа	Обзор экосист.	Консуль- тация	Защита			

Маршрут вебинара

- Типы информационных систем, уровни их защищенности
- Персональные данные данные, НПА
- SDL
- Источники данных об уязвимостях, примеры уязвимостей
- SAST, DAST
- SIEM
- Методы предотвращения угроз

Цели вебинара

1. Познакомиться с типами информационных систем с точки зрения ИБ, а также с НПА, регулирующими работу ИС, работающих с ПД
2. Познакомиться с SDL, какие требования налагаются на приложение с точки зрения ИБ
3. Узнать, откуда можно брать информацию об уязвимостях и какие средства помогают сделать приложение более безопасным

Типы информационных систем

Типы информации

Общедоступная	Общедоступная информация отдельные типы – ФЗ №149
Конфиденциальная информация	Конфиденциальная информация – ФЗ №149
Гостайна	Гос. Тайна – Закон РФ №5485-1

Характеристики информации с точки зрения ИБ

- целостность
- конфиденциалность
- доступность

Меры обеспечения информационной безопасности

- правовые
- организационные
- технические

OWASP ASVS Levels

	Applicability	Building		
Level 1	All apps		Secure Coding	Standards and checklists
Level 2	All apps	Security Architecture and Reviews	Secure Coding	Standards and checklists
Level 3	High Assurance	Security Architecture and Reviews	Secure Coding	Standards and checklists

Legend	Acceptable	Suitable

OWASP ASVS Levels

Building, Configuration, Deployment Assurance and Verification			Assurance and Verification	
Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Penetration Testing	DAST
Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST
Secure & Peer Code Review	DevSecOps	Unit and Integration Tests	Hybrid Reviews	SAST

Персональные данные

Персональные данные

- Любая информация о гражданине
  - ФИО
  - Пол, возраст
  - Образование, квалификация
  - Фото и видеозапись, которая может быть использована для его идентификации
- Защищается информация на основании ФЗ-152
- Защиту обязан производить Оператор, который обрабатывает информацию

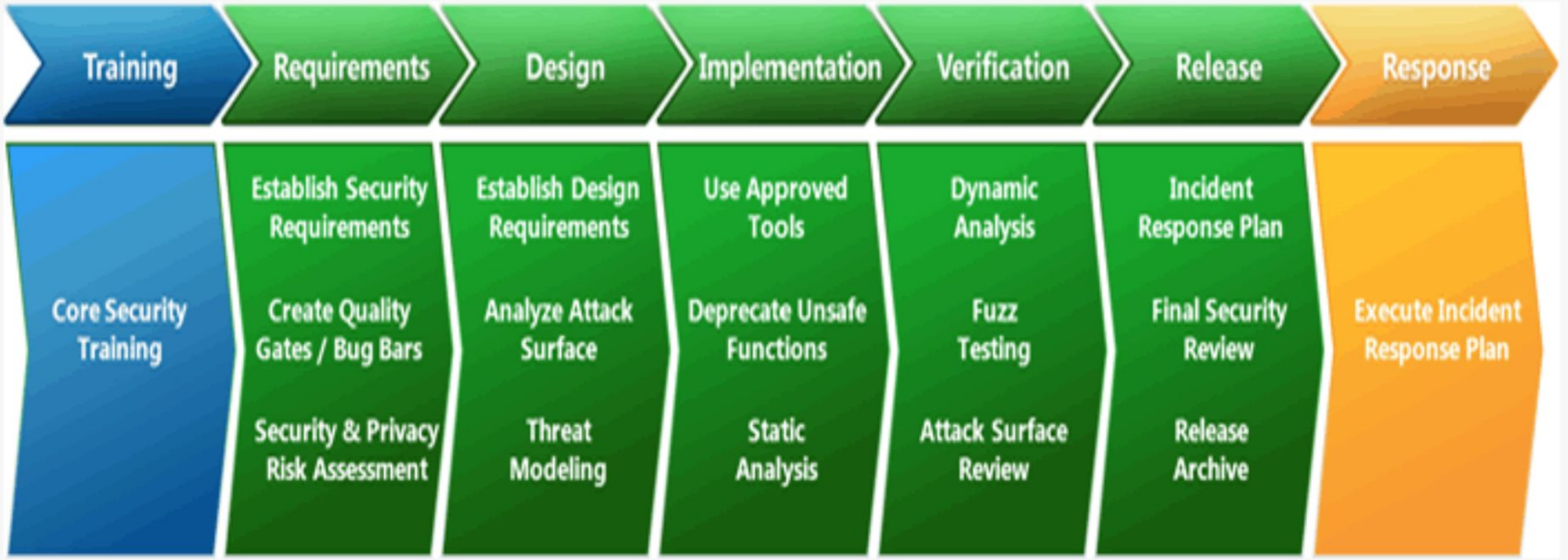
PCI DSS

- стандарт безопасности, созданный специально для защиты данных платежных карт

●https://www.nspk.ru/cards-mir/security/standart-pcidss
●https://www.pcisecuritystandards.org/document_library/?category=pcidss&documen
t=p ci_dss

Secure Development Lifecycle

Microsoft SQLC



Kotlin security code standart?

- https://www.kiuwan.com/wp-content/uploads/2022/09/Security-Guide-for-Kotlin-Developers.pdf
- https://www.stackhawk.com/blog/guide-to-security-in-kotlin/
- https://github.com/Checkmarx/Kotlin-SCP

Источники данных об уязвимостях, примеры уязвимостей

Источники данных об уязвимостях

- OWASP TOP-10 WEB, OWASP TOP-10 REST
- CWE TOP-25
- PortSwigger Web Security Academy
- https://kotlinlang.org/ docs/security.html

- CVE-2020-29582 (BDU:2022-04645) - https://nvd.nist.gov/vuln/detail/CVE-2020-29582
- CVE-2022-24329 - https://nvd.nist.gov/vuln/detail/CVE-2022-24329

Инъекции

- SQL
- HTML
- OS command
- LDAP
- XML
- …

SQL-инъекции

```
Statement statement = connect.createStatement();
String query = "SELECT user, id, pass FROM users WHERE user='" +
                user.name + "' AND pass = '" + user.pass + "'";
ResultSet resultSet = statement.executeQuery(query);
```

```
user: admin
pass: admin' or id='1
```

```
SELECT user, id, pass FROM users WHERE user='admin' AND pass = 'admin' or id='1'
```

SQL-инъекции

```
String query = "SELECT user, id, pass FROM users WHERE user=? AND pass=?";
PreparedStatement statement = connect.prepareStatement(query);
statement.setString(1, user.name);
statement.setString(2, user.pass);
ResultSet resultSet = statement.executeQuery();
```

```
user: admin
pass: admin' or id='1
```

```
SELECT user, id, pass FROM users WHERE user='admin' AND pass = 'admin\' or id=\'1'
```

SQL-инъекции

- использование raw-запросов

```
val query = "SELECT * FROM users WHERE username=$unameParam"
```

```
PreparedStatement pstmt = con.prepareStatement(
    "UPDATE EMPLOYEES SET SALARY = ? WHERE ID = ?");
pstmt.setBigDecimal(1, 153833.00)
pstmt.setInt(2, 110592)
```

SQL-инъекции

- нативный SQL в ORM

```
val users = database.useConnection { connection ->
val sql = "UPDATE users SET password=$newPassword WHERE email=$email"
val stmt = connection.createStatement()
if (stmt.execute(sql)) {
println("Password rest!")
}
}
```

```
val users = database.useConnection { connection ->
val sql = """UPDATE users SET password=? WHERE email=?"""
connection.prepareStatement(sql).use { statement ->
statement.setString(1, newPassword)
statement.setString(2, email)
statement.execute()
}
}
```

SQL-инъекции

- Ktorm - ORM для Kotlin

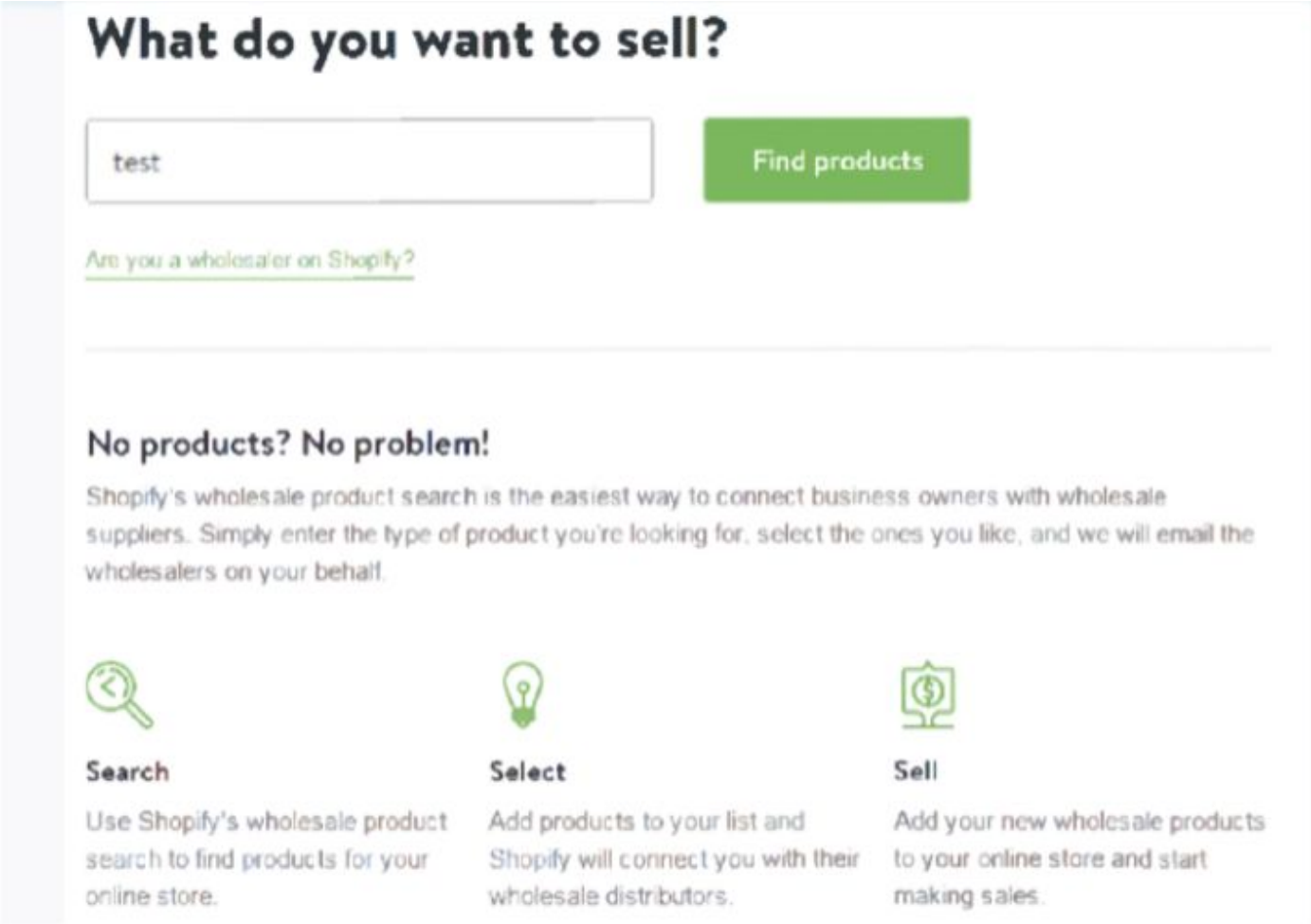
```
val database = Database.connect(
    "jdbc:mysql://localhost:3306/mydb", user = "root", password = "")

database
    .from(Users)
    .select(Users.email)
    .forEach { row ->
        println(row[Users.email])
    }
```

HTML-инъекции

- HTML
- XSS
  - reflected
  - stored
  - DOM-based

XSS

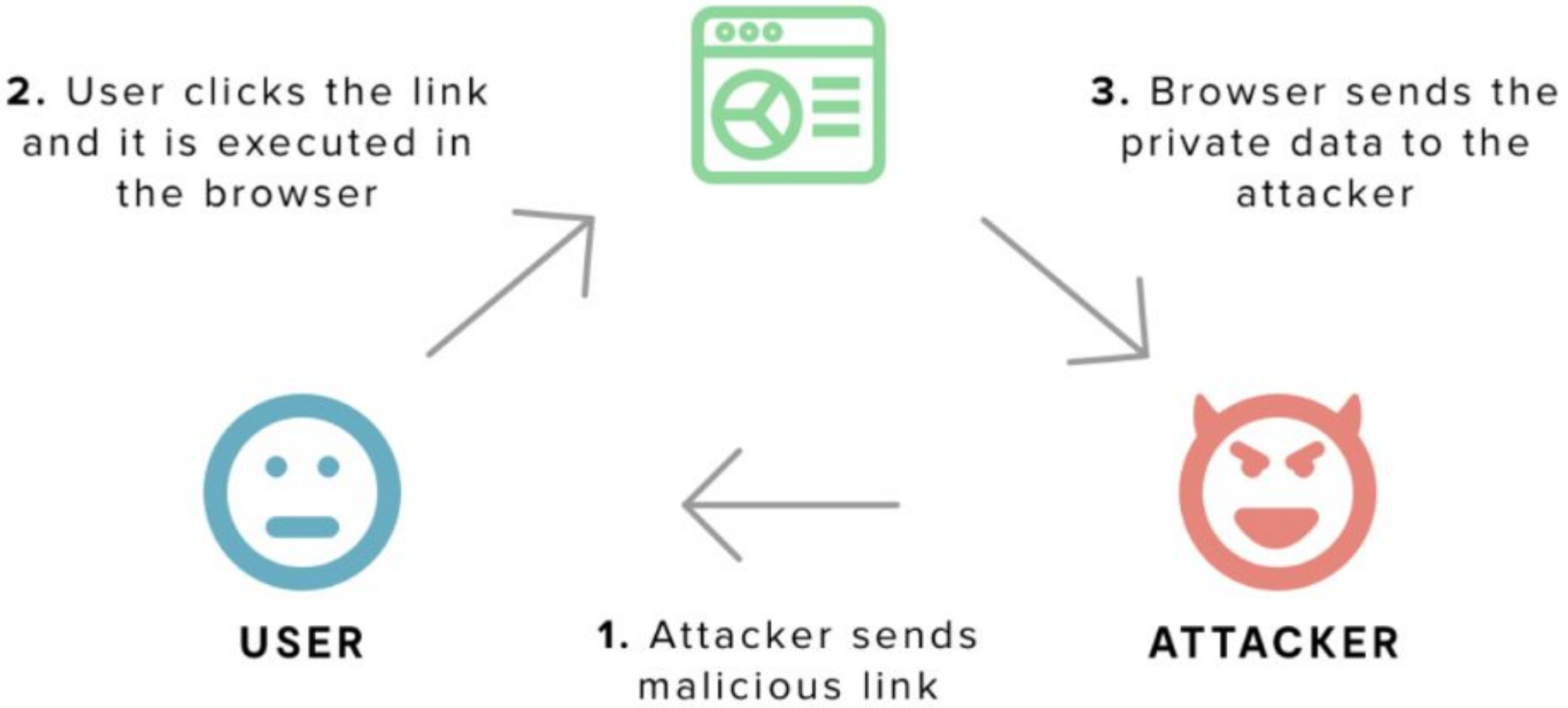


test<script>alert('XSS');</script>

XSS

vulnsite.com/search/?q=test<script>alert(%27XSS%27);</script>

XSS


2. User clicks the link and it is executed in the browser
3. Browser sends the private data to the attacker
USER
1. Attacker sends malicious link
ATTACKER

XSS-инъекции

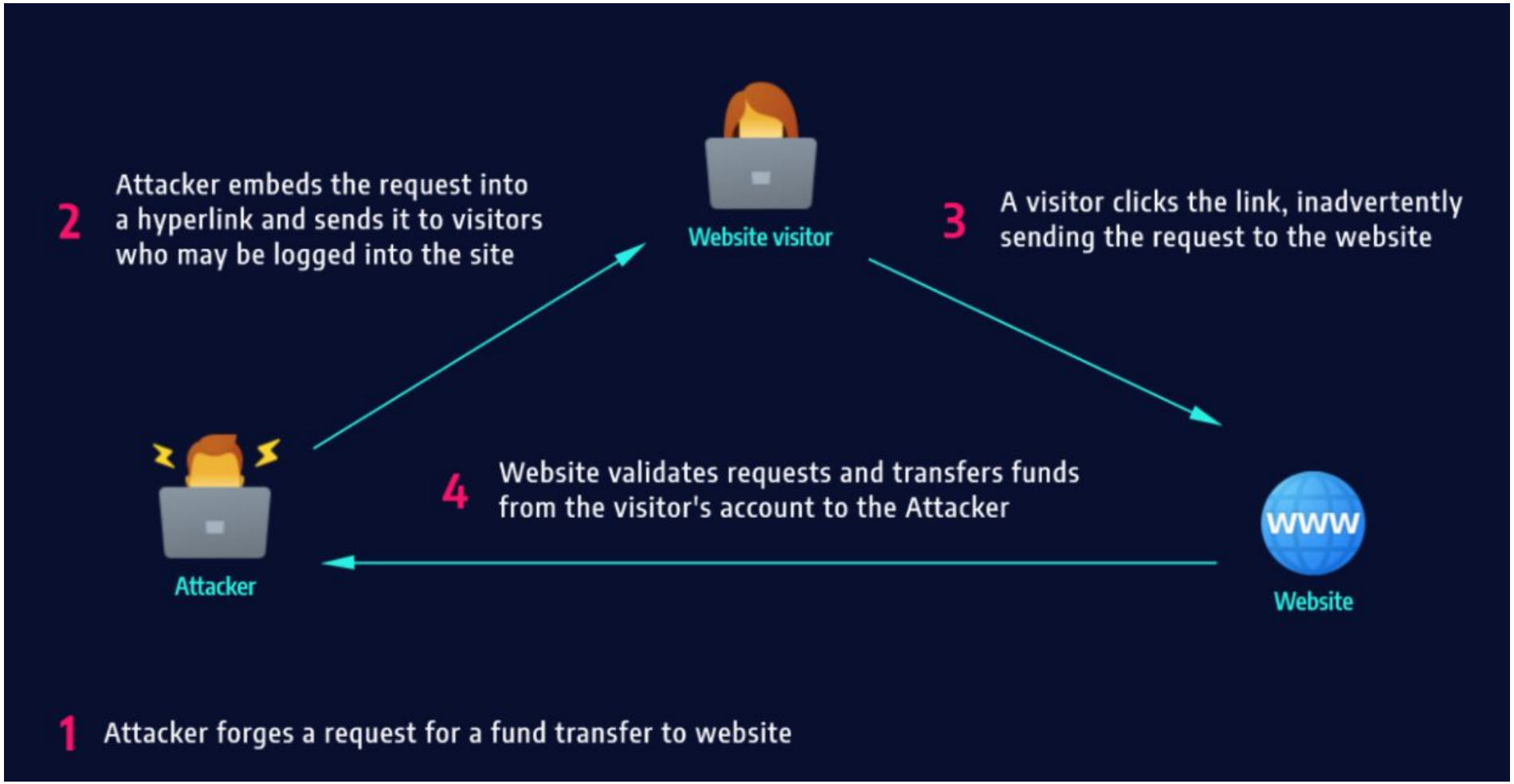
- включение url-параметра в ответ сервера

```
https://example.com/search?query=hot+dog

get("/search") {
val query = call.request.queryParameters["query"]
call.respondText("<p>Showing results for $query</p>", ContentType.Text.Html)
}
```

```
get("/search") {
val query = call.request.queryParameters["query"]
val regEx = Regex("[^A-Za-z0-9 ]")
val searchKeyWord = regEx.replace(query!!, "")
call.respondText("<p>Showing results for $searchKeyWord</p>", ContentType.Text.Html)
}
```

CSRF


2 Attacker embeds the request into a hyperlink and sends it to visitors who may be logged into the site
Website visitor
3 A visitor clicks the link, inadvertently sending the request to the website
4 Website validates requests and transfers funds from the visitor's account to the Attacker
Attacker
Website
1 Attacker forges a request for a fund transfer to website

CSRF. Рекомендации от OWASP

- предотвращение CSRF требует включения непредсказуемого токена в каждый HTTP-запрос
- предпочтительный вариант - включить уникальный токен в скрытое поле

CSRF. Защита во фреймворках

- Ktor: https://youtrack.jetbrains.com/issue/KTOR-2910/CSRF-protection-feature
- Spring: https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html

SAST, DAST и другие инструменты проверки приложения

Виды проверок программного продукта

- SAST (статический анализ)
- SCA (анализ зависимостей)
- DAST (динамический анализ)
- IAST (интерактивный анализ)
- RASP (рантайм-защита)

SAST-инструменты для языка Kotlin

- Coverity Static Analysis
- CxSAST
- Fortify
- HCL AppScan on Cloud
- Horusec
- Insider CLI
- Klocwork
- SonarCloud

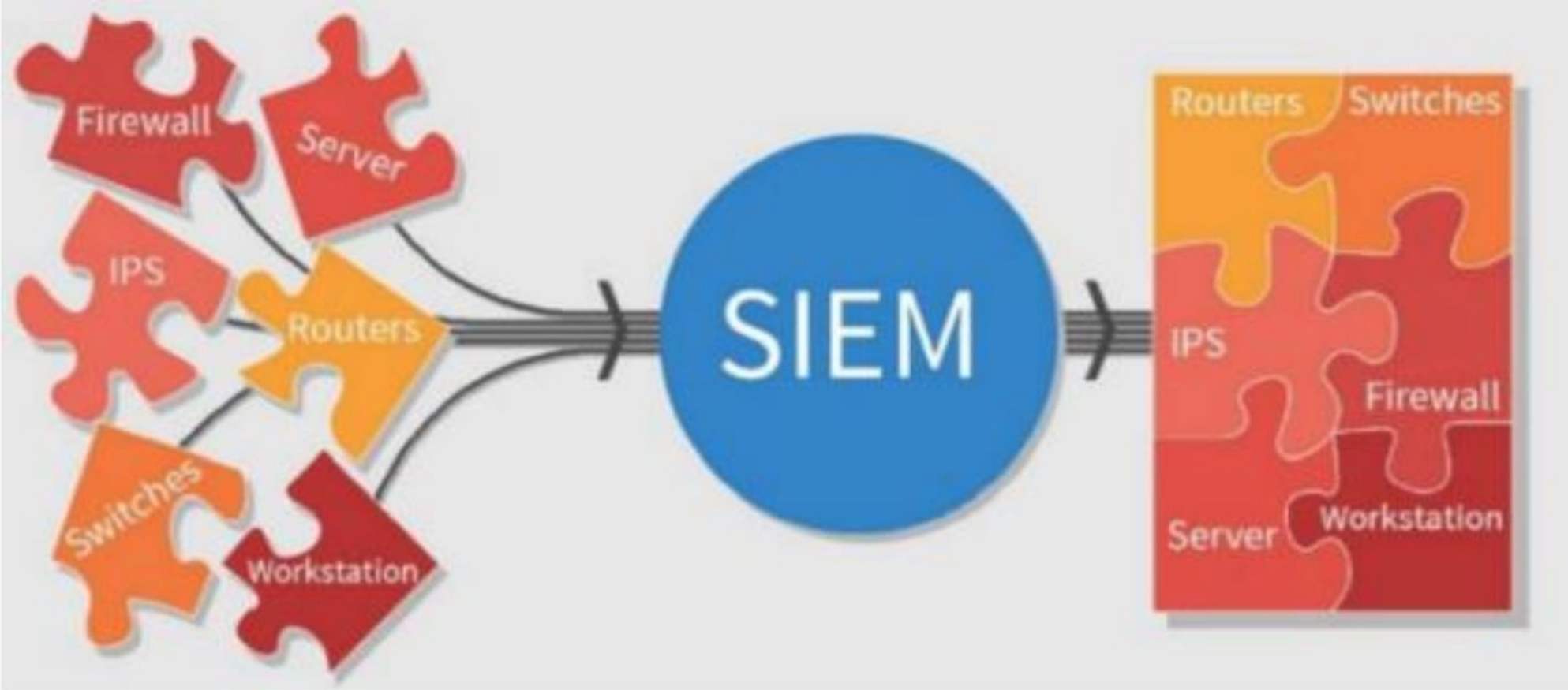
DAST-инструмены

- OWASP ZAP
- Burp Suite
- Acunetix
- Nuclei
- HCL AppScan on Cloud
- Netsparker

SIEM

Security Information and Event Management

система сбора и анализа логов с возможностью корреляции множества разнородных событий в единый инцидент



Методы предотвращения угроз

Методы и инструменты для предотвращения угроз

- инвентаризация и своевременные обновления
- сканирование имаджей
- написание кода согласно security best-practicies
- безопасное хранение секретов (hashicorp vault, Jenkins credentials, git env variables…)
- SAST, SCA, DAST + quality gates
- подпись ПО и образов
- WAF
- SIEM

Налаженные процессы:

- ревью кода
- управление доступами
- управление уязвимостями
- анализ требований законодательства

Рефлексия

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Цели вебинара

1. Познакомиться с типами информационных систем с точки зрения ИБ, а также с НПА, регулирующими работу ИС, работающих с ПД
2. Познакомиться с SDL, какие требования налагаются на приложение с точки зрения ИБ
3. Узнать, откуда можно брать информацию об уязвимостях и какие средства помогают сделать приложение более безопасным

Следующий вебинар

22.02.2024

Основы DevOps и среда выполнения приложения

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

- https://kotlinlang.org/

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары